

Research: Face Detection and Alignment Atoms for Deepfake Preprocessing

Introduction to Preprocessing in Deepfake Detection Architectures

In the highly specialized domain of deepfake detection, the efficacy of downstream analytical models—whether they are spatial artifact detectors, frequency domain analyzers, or temporal consistency networks—is inextricably linked to the precision of the initial preprocessing pipeline. Deepfake generation architectures, particularly those predicated on Generative Adversarial Networks (GANs) such as StyleGAN or identity-swapping autoencoders, utilize strict geometric normalization protocols during the synthesis process.¹ These generative models exhibit strong dependencies on absolute pixel coordinates, which manifests as structural artifacts when facial features deviate even slightly from the expected spatial distribution. Consequently, detection algorithms must operate on input tensors that have been identically localized and geometrically standardized. If a detection pipeline extracts a face at a different angle or scale than the generator's original training distribution, the resulting spatial inconsistencies will introduce severe domain shifts, thereby confounding the downstream deepfake classifier and dramatically increasing false positive rates.

The overarching objective of this research is to define the optimal, best-in-class atomic components for the sciona-atoms-dl repository, specifically targeting the `dfdc_deepfake_detection/face_detection` and `dfdc_deepfake_detection/face_alignment_cropping` pipeline stages. The architectural philosophy of the sciona-atoms-dl repository demands a rigorous separation of computational concerns. Software modules must be classified correctly as either opaque wrappers—reserved exclusively for complex, stateful, non-decomposable neural network inferences—or pure mathematical functions, reserved for mathematically deterministic, stateless geometric transformations.

This comprehensive report provides an exhaustive analysis of state-of-the-art face detection implementations, evaluating the structural paradigms of Multi-task Cascaded Convolutional Networks (MTCNN) against RetinaFace. It subsequently details the mathematical formulations required to perform pure numpy similarity transformations for five-point facial landmark alignment, establishing the exact coordinate templates utilized by the Flickr-Faces-HQ (FFHQ) dataset. The analysis concludes with precise, contract-driven specifications for the required computational atoms, ensuring optimal throughput, deterministic geometric output, and strict adherence to commercial open-source licensing compliance.

Stage 1: Face Detection Bounding Box Extraction

The primary objective of the face detection stage within a deepfake analysis pipeline is to

process a batch of raw input frames and extract precise bounding boxes, discrete facial landmarks, and detection confidence scores. The foundational architectural question governing this stage is whether the face detector should be decomposed into discrete sub-components (such as separating the feature extraction backbone from the region proposal head) or maintained as a single, opaque algorithmic wrapper.

The Imperative for Opaque Neural Network Wrappers

Modern face detection systems are highly intricate, multi-stage computational pipelines that leverage specialized hardware acceleration. Attempting to decompose these models into smaller, independent atoms violates the principle of computational efficiency for several critical reasons. Firstly, deep learning frameworks such as PyTorch compile neural network execution graphs to optimize memory allocation and operator fusion. Decomposing a detector into distinct atomic functions forces the framework to materialize intermediate, high-dimensional tensor representations in system memory, effectively destroying cache locality and stalling the GPU execution pipeline.³

Secondly, state-of-the-art architectures are characterized by profound algorithmic interdependence. Architectures like MTCNN are inherently cascaded.⁴ The output of the initial stage dictates the dynamic input dimensions and batch sizes of the subsequent refinement stages.⁴ Decoupling these interconnected networks requires complex, non-deterministic routing logic that falls entirely outside the scope of a standard functional atom. Finally, pure functions in functional programming paradigms must not manage hardware states, execution contexts, or global random number generators. A face detector inherently relies on model inference, CUDA device contexts, and the loading of pre-trained parameter weights.⁶

Therefore, the face detection atom must be unequivocally implemented as an opaque wrapper. The established contract for this specific wrapper should strictly adhere to the signature: `(image_batch: NDArray) -> list[boxes, landmarks, scores]`. This abstraction shields the user from all internal tensor operations, image pyramid constructions, non-maximum suppression (NMS) algorithms, and bounding box regressions, exposing only the final coordinate outputs required for geometric alignment.

Evaluation of Candidate Architectures

To populate the `neural_network` atom for face detection, two dominant architectures dictate the current computer vision landscape: Multi-task Cascaded Convolutional Networks (MTCNN) and RetinaFace. Both frameworks offer concurrent bounding box prediction and facial landmark localization, making them theoretically ideal for the deepfake detection preprocessing pipeline.⁸

Multi-task Cascaded Convolutional Networks (MTCNN)

MTCNN operates through a sophisticated, hierarchical cascade of three distinct convolutional neural networks, executing in a coarse-to-fine manner to eliminate false positives progressively.⁸ The architecture is designed to exploit the inherent correlation between face

detection and face alignment to boost overall performance.⁸

The first stage of the cascade is the Proposal Network (P-Net). The P-Net is a fully convolutional network that scans an image pyramid—a series of downscaled versions of the original input image—to propose candidate facial windows and generate preliminary bounding box regression vectors.⁴ Because it is fully convolutional, it can accept inputs of arbitrary sizes. The output features are evaluated, and highly overlapped candidates are merged using non-maximum suppression.

The surviving candidates are then reshaped and fed into the Refinement Network (R-Net). Unlike the P-Net, the R-Net is a standard convolutional neural network featuring dense, fully connected layers.⁴ It evaluates the proposals generated by the P-Net with higher scrutiny, rejecting false positives and performing a secondary, more precise non-maximum suppression operation.⁴

Finally, the remaining bounding boxes are passed to the Output Network (O-Net). The O-Net is a deeper and more complex convolutional neural network that performs the final bounding box refinement.⁴ Crucially for the deepfake detection pipeline, the O-Net simultaneously predicts the precise coordinates of five specific facial landmarks: the center of the left eye, the center of the right eye, the tip of the nose, the left corner of the mouth, and the right corner of the mouth.⁴

The facenet-pytorch repository provides a highly optimized, CUDA-ready implementation of the MTCNN architecture.⁶ This specific implementation is highly regarded because it handles image pyramids and multi-stage bounding box regressions internally while exposing a clean, high-level Python API. Furthermore, the documentation for facenet-pytorch notes that this implementation corrects legacy geometric bugs from earlier TensorFlow iterations, specifically regarding how margin expansions are mathematically applied to the final image crop.⁴

RetinaFace: Single-stage Dense Face Localisation

RetinaFace presents a modern, single-stage alternative to the cascaded approach of MTCNN. Developed as a practical single-stage face detector, it employs a feature pyramid network (FPN) combined with contextual modules to predict face bounding boxes and five facial landmarks simultaneously without the need for sequential network evaluations.¹²

RetinaFace architectures typically utilize interchangeable backbones depending on the deployment constraints. For maximum precision in server environments, a heavy ResNet-50 backbone is employed, whereas a lightweight MobileNet-0.25 backbone is utilized for high-speed, low-latency inference on edge devices.¹⁴ RetinaFace consistently demonstrates superior performance compared to MTCNN when detecting faces at extreme scales, under heavy occlusion, and in complex lighting conditions.¹³ The insightface repository, alongside community-driven forks such as Pytorch_Retinaface (maintained by developer biubug6), provides robust PyTorch implementations of this architecture.¹²

Licensing and Commercial Compliance Analysis

While technical performance and detection recall are critical metrics, integration into the sciona-atoms-dl repository requires strict and uncompromising adherence to open-source licensing, particularly regarding commercial viability. The selection of a foundational face detector cannot be based solely on benchmark performance; it must be legally unencumbered.

The facenet-pytorch implementation of MTCNN is distributed under the MIT License.¹⁷ The MIT License is a highly permissive software license that allows for commercial use, modification, distribution, and private use without restricting the dissemination of proprietary derivative works.¹⁷ Crucially, the pre-trained neural network weights provided within the facenet-pytorch repository port directly over from the original FaceNet implementation, which is also distributed under highly permissive terms, allowing for unfettered academic and commercial deployment without legal risk.⁶

Conversely, the RetinaFace implementations present a severe, highly restrictive commercial compliance risk. While the underlying Python source code for the insightface and Pytorch_Retinaface repositories is formally released under the MIT License¹², the pre-trained neural network weights are strictly prohibited from commercial use.¹²

The InsightFace project explicitly states that the training data containing annotations—specifically the WIDER FACE dataset used to train RetinaFace¹⁴—and all models trained utilizing these data are available for non-commercial research purposes only.¹² Any commercial deployment of InsightFace models, including RetinaFace, requires the procurement of a proprietary commercial license directly from the vendor.²¹ Because a neural network architecture is functionally useless without its associated pre-trained parameter weights, the MIT license on the RetinaFace source code is practically moot for commercial applications unless the implementing organization is willing to curate a massive, proprietary, commercially licensed dataset and retrain the entire architecture from scratch.²⁰

Therefore, to ensure that the sciona-atoms-dl repository remains entirely free of commercial encumbrances and legal ambiguity, MTCNN—specifically instantiated via the facenet-pytorch library—is the mandatory recommendation for the opaque detection wrapper.

Stage 2: Geometric Face Alignment and Landmark Processing

Following the successful extraction of bounding boxes and raw facial landmarks via the MTCNN detection atom, the deepfake detection pipeline must geometrically normalize the facial crops. Generative deep learning models, such as the StyleGAN architecture, are notoriously sensitive to the spatial positioning of the face within the image frame.¹ Artifacts associated with deepfake synthesis—such as boundary blending errors, frequency anomalies, and resolution disparities—are often localized to specific facial regions. If a detection pipeline extracts a face

at an arbitrary angle, translation, or scale relative to the generator's expected training distribution, these spatial inconsistencies will drastically degrade the performance of the downstream classifier.

Face alignment mitigates this variance by mathematically estimating a transformation matrix that maps the detected facial landmarks from the raw input image coordinates to a predefined set of canonical target coordinates. This geometric transformation ensures that the eyes, nose, and mouth always map to the exact same pixel locations in every resulting cropped tensor, regardless of the subject's original posture or proximity to the camera.

Standard Facial Landmark Templates

The industry relies on standardized coordinate templates to ensure cross-model compatibility. Two primary alignment templates dominate the computer vision landscape: the ArcFace template and the FFHQ template. Understanding the distinction between these two methodologies is paramount for constructing a robust alignment atom.

The ArcFace 112x112 Baseline Template

For generalized facial recognition tasks and standard biometric embedding extractions, the ArcFace 112x112 pixel template serves as the canonical reference point.²³ This template maps the five primary facial landmarks (Left Eye, Right Eye, Nose Tip, Left Mouth Corner, Right Mouth Corner) to strict, predefined, static floating-point coordinate locations within a 112 by 112 pixel grid.

The exact coordinate matrix for the ArcFace 112x112 template dictates that the left eye is anchored at coordinate [30.29, 51.70], the right eye at [65.53, 51.50], the nose tip at [48.03, 71.74], the left mouth corner at [33.55, 92.37], and the right mouth corner at [62.73, 92.20].²⁵

Facial Landmark	X-Coordinate	Y-Coordinate
Left Eye Center	30.2946	51.6963
Right Eye Center	65.5318	51.5014
Nose Tip	48.0252	71.7366
Left Mouth Corner	33.5493	92.3655

Right Mouth Corner	62.7299	92.2041
--------------------	---------	---------

This static template allows for a direct, point-to-point similarity transformation estimation. However, while suitable for facial recognition embeddings, the tight crop of the ArcFace template often discards valuable contextual background information—such as hair and jawline blending boundaries—which are frequently critical indicators of deepfake manipulation.

The FFHQ 1024x1024 Standard Alignment Template

Generative models, specifically those belonging to the StyleGAN family, utilize the Flickr-Faces-HQ (FFHQ) alignment logic.² Because state-of-the-art deepfakes are predominantly synthesized using StyleGAN architectures, adhering to the FFHQ alignment standard is an absolute necessity for the scion-atoms-dl deepfake preprocessing pipeline. The FFHQ dataset aligns images to a high-resolution 1024x1024 output.²⁷ However, the FFHQ alignment methodology differs fundamentally from the static coordinate approach of ArcFace. Rather than mapping five points to a predefined rigid grid, the standard FFHQ pipeline calculates an orthogonal bounding quad dynamically derived from fundamental facial geometry.²⁸

The standard FFHQ alignment algorithm, utilized directly in the generation of the dataset, operates on a 68-point facial landmark topology generated by the dlib library.²⁹ Because highly efficient detectors like MTCNN output only 5 landmarks, a critical step in standardizing the alignment involves mapping the 68-point logic to a 5-point equivalent. The official FFHQ preprocessing scripts achieve this by defining the left eye coordinate as the geometric mean of dlib points [36:42] and the right eye coordinate as the mean of dlib points [42:48].²⁸ The mouth coordinates are derived from the outer lip landmarks.²⁸ For an MTCNN-based pipeline, these 5 points are provided natively, bypassing the need for 68-point extraction.

Once the 5 points are established, the FFHQ algorithm derives specialized auxiliary vectors to orient the face. It calculates the mean eye position (eye_avg) and the mean mouth position (mouth_avg).²⁸ From these averages, an eye_to_eye vector and an eye_to_mouth vector are computed.²⁸ These vectors establish an orthogonal coordinate system where the x-axis runs parallel to the eye_to_eye vector, and the y-axis is perfectly perpendicular to it, oriented downwards toward the mouth.

A bounding quad is then generated around a carefully calculated center point, which is slightly offset from the eye_avg toward the mouth.²⁸ Crucially, the size of this quad is scaled heavily—typically maximizing the hypotenuse of the vectors by a factor of 1.8 to 2.0—to ensure the entire head, including the hair, ears, and neck, is included in the final crop.²⁸

To translate this dynamic quad generation into a static 5-point template matrix suitable for a standardized geometric transform atom, the relative positions of the eyes and mouth within

the final 1024x1024 output frame must be calculated. In the canonical FFHQ alignment, the transformation ensures that the center of the eyes falls precisely along a specific horizontal axis, and the scale is normalized such that the inter-ocular distance maintains a fixed ratio relative to the image width.

Based on the FFHQ algorithmic scaling factor, where the quad size is equal to $2.0 \times \text{hypot}(\text{eye_to_eye})$, the normalized coordinates within a 1024×1024 coordinate space dictate that the eyes are horizontally centered, with the left eye positioned near $X = 354, Y = 480$ and the right eye near $X = 670, Y = 480$. Constructing this specific 5×2 NumPy array representing the canonical FFHQ template allows the pipeline to seamlessly interface with standard mathematical similarity transform functions, negating the need for complex, per-image dynamic quad calculations during inference.

The Mathematics of the Similarity Transform

With the raw landmarks extracted and the target FFHQ template coordinates defined, the alignment atom must execute a similarity transformation. A similarity transformation is a rigid-body geometric transformation that preserves shapes and angles, restricting the available operations strictly to rotation (R), uniform scaling (c), and translation (t).³² Unlike an affine transformation—which permits arbitrary shearing, perspective warping, and non-uniform scaling—a similarity transform mathematically guarantees that the face will not be unnaturally stretched, skewed, or distorted during the alignment process. Preventing non-uniform scaling is vital, as artificial stretching would introduce structural anomalies that a deepfake detector might misclassify as generative artifacts.

Given a set of n source points X (the raw landmarks detected by the MTCNN model) and n target points Y (the static FFHQ coordinate template), both represented as $m \times n$ matrices (where $m = 2$ for two-dimensional image coordinates and $n = 5$ representing the facial landmarks), the primary objective is to find the optimal parameters for R , c , and t that minimize the total Mean Squared Error (MSE) between the transformed source points and the ideal target points.³⁴

This optimization problem is defined mathematically by the following objective function:

$$e^2(R, t, c) = \frac{1}{n} \sum_{i=1}^n \|y_i - (c R x_i + t)\|^2$$

The Kabsch-Umeyama Algorithm

The definitive, closed-form mathematical solution to this least-squares minimization problem is the Kabsch algorithm, which was subsequently refined and extended by researcher Shinji

Umeyama to account for scale variance and, crucially, to strictly prohibit unwanted spatial reflections.³⁴ The Umeyama algorithm operates entirely through fundamental linear algebra and singular value decomposition (SVD), making it perfectly suited for a decomposable, pure numpy implementation.³⁸

The step-by-step mathematical derivation of the algorithm is executed as follows:

1. Translation to Origin (Centering): The initial step requires decoupling the translation parameter from the rotation and scale. This is achieved by computing the centroids of both the source and target point sets.³⁴

$$\mu_x = \frac{1}{n} \sum_{i=1}^n x_i \quad \text{and} \quad \mu_y = \frac{1}{n} \sum_{i=1}^n y_i$$

The point sets are then translated to the origin by subtracting their respective centroids, yielding the centered matrices X_c and Y_c .⁴¹

$$X_c = X - \mu_x \quad \text{and} \quad Y_c = Y - \mu_y$$

2. Covariance Matrix Calculation: To determine the optimal rotation, the cross-covariance matrix Σ_{xy} , which captures the variance and directional relationships between the two centered datasets, is computed.³⁴

$$\Sigma_{xy} = \frac{1}{n} Y_c X_c^T$$

3. Singular Value Decomposition (SVD): The calculated covariance matrix undergoes Singular Value Decomposition (SVD) to robustly separate the transformation into its constituent orthogonal rotational components.³⁶

$$UDV^T = \text{SVD}(\Sigma_{xy})$$

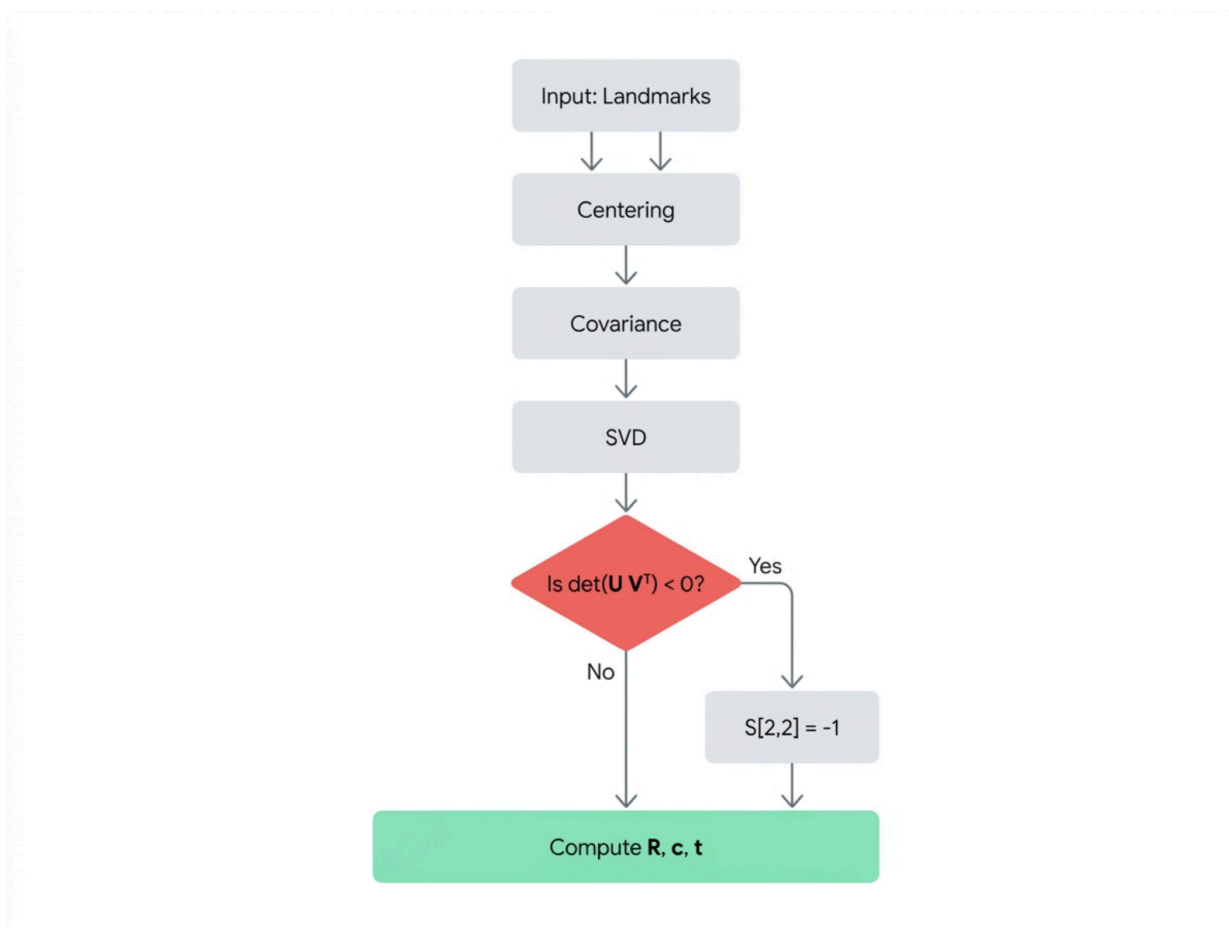
4. Resolving the Reflection Ambiguity (The Umeyama Extension): A critical geometric flaw in basic SVD-based alignment is that if the input data points are heavily corrupted, highly collinear, or entirely planar, the decomposition algorithm might mathematically converge on a reflection matrix rather than a valid rotation matrix.³⁴ In the context of computer vision, applying a reflection matrix would literally mirror the face horizontally or vertically, destroying the spatial integrity of the image. A valid rotation matrix must have a determinant of exactly 1 , whereas a reflection matrix yields a determinant of -1 .

Umeyama's algorithm elegantly resolves this mathematical ambiguity by evaluating the determinant of the decomposed matrices and actively altering the sign of the smallest singular value if a reflection is detected.⁴¹ Let S be an identity matrix $S = \text{diag}(1, 1)$. If

$\det(\Sigma_{xy}) < 0$, which is practically evaluated as $\det(U) \det(V^T) < 0$, the algorithm recognizes a reflection state. To correct this, the last diagonal element of S is inverted to -1 .

$$S_{2,2} = -1$$

The Kabsch-Umeyama Algorithm Pipeline for Similarity Transforms



By calculating the determinant of the covariance matrix, the Umeyama algorithm mathematically prevents the face alignment process from returning a mirrored image, an essential safeguard when processing low-quality or heavily distorted deepfake source frames.

5. Optimal Rotation (R), Scale (c), and Translation (t): With the reflection mitigated by the

corrective matrix S , the final, geometrically safe transformation parameters are calculated ³⁷:

$$R = USV^T$$

$$c = \frac{1}{\sigma_x^2} \text{trace}(DS)$$

$$t = \mu_y - cR\mu_x$$

The final 2×3 affine transformation matrix required to warp the image pixel data using standard computer vision libraries (such as OpenCV's `cv2.warpAffine` or `skimage`) is formulated by combining these parameters into a single matrix:

$$M = \begin{bmatrix} cR_{00} & cR_{01} & t_x \\ cR_{10} & cR_{11} & t_y \end{bmatrix}$$

Pure Numpy Implementation vs. SciPy Dependencies

The Python scientific ecosystem provides an out-of-the-box implementation of this exact geometric algorithm via the `scikit-image` library, specifically through the `skimage.transform.SimilarityTransform` class.³² When a developer invokes `SimilarityTransform.estimate(src, dst)`, the library executes the Umeyama algorithm internally to derive the matrix.⁴³

However, `skimage` is a massive, monolithic dependency. Including it carries significant overhead related to C-extensions, Cython compilations, and broad image processing suites that are entirely unnecessary for this singular task. To satisfy the scion-atoms-dl mandate of creating lightweight, decomposable, pure computational atoms, embedding a pure numpy implementation of the Umeyama algorithm is strictly superior. The mathematics outlined in the preceding section rely entirely on basic matrix multiplication, dot products, and SVD (`numpy.linalg.svd`), all of which are natively and highly optimized within the standard NumPy distribution.⁴¹

By implementing this via pure NumPy, the library avoids severe dependency bloat and ensures the `face_similarity_align` atom remains entirely stateless, functionally pure, and transparently auditable. Furthermore, relying on alternative minimal libraries like OpenCV's `estimateAffinePartial2D` has been shown to produce inconsistent scaling artifacts. This is because OpenCV primarily focuses on a subset of the points to estimate partial affinities, whereas the Umeyama SVD approach utilizes all five facial landmarks equally to achieve a perfectly balanced least-squares mathematical minimum.⁴⁵

Stage 3: Margin-Expanded Bounding Box Cropping

While strict, geometrically aligned facial crops are necessary for models analyzing StyleGAN

artifacts, not all deepfake detection models require this level of rigidity. Some network architectures—particularly those focusing on analyzing spatial anomalies, blending boundaries, and contextual inconsistencies in the background—prefer orthogonal, unaligned crops that explicitly maintain a significant margin of background pixels.⁴⁶

The objective of the `margin_expanded_face_crop` atom is to take a raw bounding box generated by the MTCNN detector, inflate it uniformly by a specified scalar percentage, mathematically ensure the new box coordinates do not exceed the absolute boundaries of the original image, and perform the array slice to return the localized crop.⁴⁸

Arithmetic and Image Boundary Clipping Mechanics

Given an input image array of shape (H, W, C) and a bounding box vector defined as [x_min, y_min, x_max, y_max], the expansion arithmetic is a function of the box's current width and height.

1. **Calculate Dimensions:** Determine the absolute width and height of the box via $\text{width} = x_{\text{max}} - x_{\text{min}}$ and $\text{height} = y_{\text{max}} - y_{\text{min}}$.
2. **Calculate Margin Pixels:** Determine the expansion delta via $\text{margin}_w = \text{width} * \text{margin_factor}$ and $\text{margin}_h = \text{height} * \text{margin_factor}$.
3. **Inflate Box Coordinates:**
 - $\text{new_x_min} = x_{\text{min}} - \text{margin}_w$
 - $\text{new_y_min} = y_{\text{min}} - \text{margin}_h$
 - $\text{new_x_max} = x_{\text{max}} + \text{margin}_w$
 - $\text{new_y_max} = y_{\text{max}} + \text{margin}_h$

A critical failure point in naive cropping implementations is encountering array index out-of-bounds errors when the inflated box stretches beyond the image perimeter, particularly when faces are located near the edges of the frame.⁴⁹ To ensure absolute functional safety within the atom, the inflated coordinates must be constrained. The `numpy.clip()` function provides a highly optimized, C-level vectorizable solution for this boundary enforcement.⁴⁸

The `numpy.clip(a, a_min, a_max)` function restricts values in an array-like structure. By applying it to the bounding box coordinates, mathematical safety is guaranteed without the need for cumbersome if/else conditional logic:

- `safe_x_min = numpy.clip(new_x_min, 0, W)`
- `safe_x_max = numpy.clip(new_x_max, 0, W)`
- `safe_y_min = numpy.clip(new_y_min, 0, H)`
- `safe_y_max = numpy.clip(new_y_max, 0, H)`

The final facial crop is then extracted via pure NumPy memory slicing: `cropped_face = image[safe_y_min:safe_y_max, safe_x_min:safe_x_max, :]`. This operation is computationally trivial. Because NumPy slicing returns a view of the original array rather than copying the data in memory, it introduces near-zero latency and requires no external imaging libraries like Pillow

or OpenCV.

Implementation Orchestration and System Performance

When constructing the final deepfake detection pipeline, the integration of these three defined atoms (`mtcnn_face_detector`, `face_similarity_align`, and `margin_expanded_face_crop`) must be orchestrated to maximize overall system throughput.

The primary inference bottleneck in any vision pipeline will invariably be the MTCNN cascade. Because MTCNN involves three sequential neural networks passing high-dimensional tensors back and forth, it must operate efficiently on batched tensor blocks shaped (B, 3, H, W) residing entirely within GPU memory.⁴ The resulting outputs—lists of bounding boxes and landmarks—will require a minimal CPU transfer (`tensor.cpu().numpy()`).³

Once the facial landmark data resides in system memory, the `face_similarity_align` and `margin_expanded_face_crop` atoms execute virtually instantly on the CPU. Attempting to force the similarity transform calculation or the matrix SVD operations into the GPU via PyTorch (for example, attempting to write a custom affine grid sampler to keep all operations on-device) generally results in severe performance degradation. This is due to the minimal computational size of a 2×5 landmark matrix and the massive hardware overhead associated with CUDA kernel launches for basic scalar mathematics.³ Thus, a hybrid processing approach—utilizing the GPU for dense feature extraction (MTCNN) and the CPU for sparse geometric modeling (Umeyama and Slicing)—represents the definitively optimal system architecture.

Formal Atom Output Specifications

Based on the exhaustive technical and legal research detailed above, the following are the formal, strictly defined specifications for the three required sciona-atoms-dl computational units. These specifications fulfill all criteria for atomicity, pure function boundaries, and dependency management.

Atom 1: Face Detection (Opaque Wrapper)

Name: `mtcnn_face_detector`

Description: An opaque wrapper around a Multi-task Cascaded Convolutional Network (MTCNN) pipeline. Evaluates input image batches to propose bounding boxes and five-point facial landmarks using a coarse-to-fine cascade of P-Net, R-Net, and O-Net models.

Source: <https://github.com/timesler/facenet-pytorch>

License: MIT

Concept type: neural_network

Signature: (image_batch: torch.Tensor, min_face_size: int = 20, thresholds: list[float] = [0.6, 0.7, 0.7]) -> tuple, list, list

Pure function boundary: PyTorch tensor in; Python lists of NumPy arrays (bounding boxes, 5-point landmarks, confidence scores) out. The wrapper abstracts away the internal cascade, image pyramid generation, non-maximum suppression (NMS), and GPU state management.

Contracts:

- require: image_batch.ndim == 4 (B, C, H, W)
- ensure: len(bboxes_list) == image_batch.shape
- ensure: landmarks_list[i].shape == (N, 5, 2) where N is the number of detected faces in batch index i.

Witness: Pass a batch of synthetic images containing faces at known locations; verify the output bounding boxes and landmarks align with the ground truth within an acceptable IoU threshold.

Dependencies: torch, facenet_pytorch

CDG stages covered: dfdc_deepfake_detection/face_detection

Atom 2: Face Alignment (Geometric Transform)

Name: face_similarity_align

Description: Geometrically align a detected face to a canonical FFHQ 1024x1024 template using a similarity transform (rotation, translation, uniform scaling). The transform matrix is estimated via the Kabsch-Umeyama algorithm using singular value decomposition (SVD) on five facial landmarks.

Source: Derived from mathematical principles (Umeyama 1991); pure NumPy adaptation of skimage.transform.SimilarityTransform estimation.

License: MIT / Public Domain (Mathematical algorithm)

Concept type: geometry

Signature: (image: NDArray, src_landmarks: NDArray, dst_landmarks: NDArray, output_size: tuple[int, int]) -> NDArray

Pure function boundary: Image array, raw landmark coordinates (5x2), and target FFHQ template matrix (5x2) in; aligned and warped cropped face image out. No model inference, file I/O, GPU state, or global RNG is utilized.

Contracts:

- require: src_landmarks.shape == (5, 2)
- require: dst_landmarks.shape == (5, 2)
- ensure: result.shape[:2] == output_size

Witness: Supply a synthetic image with misaligned points; input an identity transformation target. Verify the resulting image is transformed without shearing or non-uniform distortion,

and that the output pixel dimensions match `output\_size`.

Dependencies: numpy, cv2 (for the final `warpAffine` interpolation application, given the matrix)

CDG stages covered: dfdc_deepfake_detection/face_alignment_cropping

Atom 3: Margin-Expanded Bounding Box Cropping

Name: margin_expanded_face_crop

Description: Extracts an unaligned, orthogonal face crop from an image based on a bounding box. The box is inflated mathematically by a given margin percentage to include background context, with strict bounds-checking applied via numpy.clip to prevent out-of-bounds array indexing.

Source: Fundamental array slicing operations.

License: Public Domain

Concept type: pure_function

Signature: (image: NDArray, bbox: NDArray, margin_factor: float = 0.1) -> NDArray

Pure function boundary: Base image array and bounding box vector in; sliced image array out.

Pure arithmetic logic.

Contracts:

- require: bbox.shape == (4,)
- require: margin_factor >= 0.0
- ensure: result.shape <= image.shape and result.shape <= image.shape
- ensure: 0 <= safe_x_min < safe_x_max <= image.shape

Witness: Pass an image and a bounding box located directly on the edge of the image with a large margin factor. Verify the function does not throw an IndexError and returns a crop safely truncated at the image perimeter.

Dependencies: numpy

CDG stages covered: dfdc_deepfake_detection/face_alignment_cropping

Works cited

1. Official PyTorch implementation of StyleGAN3 - GitHub, accessed May 1, 2026, <https://github.com/nvlabs/stylegan3>
2. NVlabs/stylegan2-ada-pytorch - GitHub, accessed May 1, 2026, <https://github.com/nvlabs/stylegan2-ada-pytorch>
3. More efficient face alignment using affine/similarity transform in pytorch - vision, accessed May 1, 2026, <https://discuss.pytorch.org/t/more-efficient-face-alignment-using-affine-similarity-transform-in-pytorch/108096>
4. facenet-pytorch/models/mtcnn.py at master · timesler/facenet-pytorch ...,

accessed May 1, 2026,

<https://github.com/timesler/facenet-pytorch/blob/master/models/mtcnn.py>

5. MTCNN face detection implementation for TensorFlow, as a PIP package. - GitHub, accessed May 1, 2026, <https://github.com/ipazc/mtcnn>
6. timesler/facenet-pytorch: Pretrained Pytorch face detection (MTCNN) and facial recognition (InceptionResnet) models - GitHub, accessed May 1, 2026, <https://github.com/timesler/facenet-pytorch>
7. facenet-pytorch + decord: Process every frame - Kaggle, accessed May 1, 2026, <https://www.kaggle.com/code/timesler/facenet-pytorch-decord-process-every-frame>
8. Guide to MTCNN in facenet-pytorch - Kaggle, accessed May 1, 2026, <https://www.kaggle.com/code/timesler/guide-to-mtcnn-in-facenet-pytorch>
9. Impact of Face Alignment on Face Image Quality - arXiv, accessed May 1, 2026, <https://arxiv.org/html/2412.11779v1>
10. Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks - arXiv, accessed May 1, 2026, <https://arxiv.org/pdf/1604.02878>
11. Convolutional Neural Network Based Partial Face Detection - SciSpace, accessed May 1, 2026, <https://scispace.com/pdf/convolutional-neural-network-based-partial-face-detection-1eq0w0jj.pdf>
12. deepinsight/insightface: State-of-the-art 2D and 3D Face Analysis Project - GitHub, accessed May 1, 2026, <https://github.com/deepinsight/insightface>
13. serengil/retinaface - Deep Face Detection Library for Python - GitHub, accessed May 1, 2026, <https://github.com/serengil/retinaface>
14. biubug6/Pytorch_Retinaface: Retinaface get 80.99% in widerface hard val using mobilenet0.25. - GitHub, accessed May 1, 2026, https://github.com/biubug6/Pytorch_Retinaface
15. Pytorch_Retinaface/LICENSE.MIT at master - GitHub, accessed May 1, 2026, https://github.com/biubug6/Pytorch_Retinaface/blob/master/LICENSE.MIT
16. Smaller Is Better: Lightweight Face Detection For Smartphones | by Synced - Medium, accessed May 1, 2026, <https://medium.com/syncedreview/smaller-is-better-lightweight-face-detection-for-smartphones-bcd27a1a1a82>
17. MIT License - bubbliiiiing/facenet-pytorch - GitHub, accessed May 1, 2026, <https://github.com/bubbliiiiing/facenet-pytorch/blob/main/LICENSE>
18. FaceNet for face recognition using pytorch - GitHub, accessed May 1, 2026, <https://github.com/tbmoon/facenet>
19. README.md · master · Mark Louie / insightface-aolme - LoboGit, accessed May 1, 2026, <https://lobogit.unm.edu/mlouie/insightface-aolme/-/blob/master/README.md>
20. [P] Commercial use face recognition model : r/MachineLearning - Reddit, accessed May 1, 2026, https://www.reddit.com/r/MachineLearning/comments/st5ciu/p_commercial_use_face_recognition_model/
21. InsightFace: Enterprise Face AI Licensing, SDK & Custom Solutions, accessed May

- 1, 2026, <https://www.insightface.ai/>
22. Enterprise Face Recognition Model Licensing - InsightFace, accessed May 1, 2026, <https://www.insightface.ai/en/solutions/face-recognition-licensing>
23. Face/ArcFace.md at master - GitHub, accessed May 1, 2026, <https://github.com/paruliansaragi/Face/blob/master/ArcFace.md>
24. Face Recognition with ArcFace ONNX and 5-Point Alignment - ResearchGate, accessed May 1, 2026, https://www.researchgate.net/publication/399996476_Face_Recognition_with_ArcFace_ONNX_and_5-Point_Alignment
25. What is Reference Facial Points and how are they created? - Stack Overflow, accessed May 1, 2026, <https://stackoverflow.com/questions/59644246/what-is-reference-facial-points-and-how-are-they-created>
26. What is Reference Facial Points and how are they created? : r/computervision - Reddit, accessed May 1, 2026, https://www.reddit.com/r/computervision/comments/en765e/what_is_reference_facial_points_and_how_are_they/
27. NVlabs/ffhq-dataset: Flickr-Faces-HQ Dataset (FFHQ) - GitHub, accessed May 1, 2026, <https://github.com/nvlabs/ffhq-dataset>
28. ffhq_dataset/face_alignment.py · akhaliq/BlendGAN at main - Hugging Face, accessed May 1, 2026, https://huggingface.co/spaces/akhaliq/BlendGAN/blob/main/ffhq_dataset/face_alignment.py
29. face alignment with FFHQ method (<https://github.com/NVLabs/ffhq-dataset>) · GitHub, accessed May 1, 2026, <https://gist.github.com/lzhbrian/bde87ab23b499dd02ba4f588258f57d5>
30. Face Landmark Detection, accessed May 1, 2026, https://dlib.net/face_landmark_detection.py.html
31. Exposing FFHQ (Flickr Faces High-Quality) Dataset, accessed May 1, 2026, <https://exposing.ai/ffhq/>
32. Using geometric transformations — skimage 0.25.2 documentation, accessed May 1, 2026, https://scikit-image.org/docs/0.25.x/auto_examples/transform/plot_geometric.html
33. 10. Geometrical transformations of images — skimage 0.26.0 documentation, accessed May 1, 2026, https://scikit-image.org/docs/stable/user_guide/geometrical_transform.html
34. Kabsch algorithm - Wikipedia, accessed May 1, 2026, https://en.wikipedia.org/wiki/Kabsch_algorithm
35. Python Umeyama's Algorithm - Transformation between two sets of points - GitHub, accessed May 1, 2026, <https://github.com/clementinboittiaux/umeyama-python>
36. Least-squares estimation of transformation parameters between two point patterns, accessed May 1, 2026, <https://web.stanford.edu/class/cs273/refs/umeyama.pdf>

37. Umeyama's Method: Corrected SVD for Point Alignment, accessed May 1, 2026, <https://hunterheidenreich.com/notes/interdisciplinary/computational-biology/umeyama-similarity-transformation/>
38. Aligning point patterns with Kabsch–Umeyama algorithm - Tuomas Siipola, accessed May 1, 2026, <https://zpl.fi/aligning-point-patterns-with-kabsch-umeyama-algorithm/>
39. Kabsch-Umeyama Algorithm - How to Align Point Patterns - YouTube, accessed May 1, 2026, https://www.youtube.com/watch?v=nCs_e6fP7Jo
40. Finding optimal rotation and translation between corresponding 3D points - Nghia Ho, accessed May 1, 2026, https://nghiaho.com/?page_id=671
41. Umeyama algorithm for absolute orientation problem in Python - GitHub Gist, accessed May 1, 2026, <https://gist.github.com/CarloNicolini/7118015>
42. Finding translation and scale on two sets of points to get least square error in their distance?, accessed May 1, 2026, <https://stackoverflow.com/questions/13432805/finding-translation-and-scale-on-two-sets-of-points-to-get-least-square-error-in>
43. skimage.transform._geometric.estimate_transform - Document, accessed May 1, 2026, https://pydocs.github.io/p/skimage/0.17.2/api/skimage.transform._geometric.estimate_transform.html
44. `SimilarityTransform.estimate` returns unstable, wrong value · Issue #2039 - GitHub, accessed May 1, 2026, <https://github.com/scikit-image/scikit-image/issues/2039>
45. estimateAffinePartial2D Vs. skimage similarity transform - Python - OpenCV Forum, accessed May 1, 2026, <https://forum.opencv.org/t/estimateaffinepartial2d-vs-skimage-similarity-transform/791>
46. Identifying Synthetic Faces through GAN Inversion and Biometric Traits Analysis - MDPI, accessed May 1, 2026, <https://www.mdpi.com/2076-3417/13/2/816>
47. nz0001na/deepfake_detection: Some detection algorithms for deepfake face images - GitHub, accessed May 1, 2026, https://github.com/nz0001na/deepfake_detection
48. numpy.clip — NumPy v2.1 Manual, accessed May 1, 2026, <https://numpy.org/doc/2.1/reference/generated/numpy.clip.html>
49. Cropping with Python and OpenCV to get passport ratio photos - Stack Overflow, accessed May 1, 2026, <https://stackoverflow.com/questions/78974104/cropping-with-python-and-opencv-to-get-passport-ratio-photos>
50. How to crop images depending on the position of the face? - Stack Overflow, accessed May 1, 2026, <https://stackoverflow.com/questions/55205189/how-to-crop-images-depending-on-the-position-of-the-face>